

DECODE SIH 2026



**PRESENTED
BY:
PRAVAH**

PRAVAH
Quantise the flow,Orchestrate the city

TEAM MEMBERS

Six undergrads from Delhi Technological University, one system to build.

- **Tanmay Goyal, Team Lead.** Heads the deep learning and reinforcement learning work, the brain that decides how traffic gets controlled. (CSE)
- **Akshit Garg, Simulation Lead.** Builds and maintains the live SUMO twin, keeping the virtual city in sync with the real one. (ECE)
- **Maulik Jain, Hardware Head.** Owns the physical side, the cameras, signals and mid-road control points that put PRAVAH on the ground. (Mechanical)
- **Abhay Singh, Computer Vision Lead.** Runs the camera pipeline that detects, tracks and classifies every vehicle at the junction. (Software Engineering)
- **Shourya Sharma, Actuation Lead.** Turns the system's decisions into smooth, real-world signal action without disrupting the flow. (Mathematics and Computing)
- **Reet Kaur Saini, Data and Sensors Lead.** Handles the GNSS and sensor feeds, making sure clean, reliable data reaches the system. (Electrical)

PROBLEM STATEMENT

Problem Title : PS1: AI Traffic Digital Twin for smart traffic optimization and congestion prediction.

What does this Problem Aim to Solve?

City traffic is growing every year, but there is still no complete, end to end system that manages it intelligently. What cities need is a system that handles rising traffic before it becomes a problem, not after. This matters because congestion is very hard to undo once it forms. By the time a jam has built up, almost nothing can clear it quickly. So, the real goal should be to prevent congestion from forming in the first place, rather than trying to dissolve it after the damage is done.

The trouble usually starts with a small bottleneck. Traffic keeps arriving and piling up at that point, and vehicles accumulate even on stretches where the flow should have been easing off. It behaves like water in a partly blocked drain. As long as more keeps coming in, it eventually overflows, and a jam appears even in places where one should never have formed.

PROBLEM STATEMENT

Why did your team choose this?

Traffic congestion is not a niche infrastructure issue. It is a daily problem faced by each one of us. We sit in it, our families sit in it, and emergency vehicles get stuck in it. Indian cities **lose an estimated \$22 billion every year to congestion** through wasted fuel, lost working hours and avoidable pollution. The number of vehicles on the road grows every year but the roads do not, and without a smarter way to manage flow, the problem only gets worse.

Despite all the technology available today, there is still no complete, end to end system that manages city traffic intelligently. What exists is patchy, short sighted and reactive. **Systems try to clear jams after they have formed, which is nearly impossible to do well.** The real opportunity, and the reason we took up this problem, is to build a framework that prevents congestion from forming in the first place rather than struggling to dissolve it after the damage is done.

REAL-WORLD PROBLEM ALIGNMENT

Who faces this issue? (E.G., Rural Students, Local Governance, Citizens, etc.)

Every person on the road. A school child whose thirty minute bus ride stretches to over an hour. **A working professional** who left on time but still walks into office late because the road would not move. An **elderly person** in a car with family, exhausted by a trip that should have been simple. And a **patient in an ambulance**, where every minute stuck in traffic can mean the difference between life and death. **Congestion does not choose who it slows down. If you are on the road, you are in it.**

REAL-WORLD PROBLEM ALIGNMENT

What's the current gap in the system?

- Today's traffic systems **are patchy and short sighted**. Most rely on cameras that can only see what is right in front of the junction. They **react to congestion** after it has already arrived instead of seeing it build up from a distance. Cities need a longer range view of traffic so the system can **prepare in advance, not respond at the last moment**.
- Current systems also **treat every vehicle the same**. **Heavy vehicles** like **trucks and buses** take much longer to start moving and **waste far more fuel when stopped repeatedly**. Signals give them **no extra allowance**, so the light turns red before the pack clears and the junction never reaches its best flow.
- **Emergency vehicles** are **not truly prioritised** either. **Ambulances and fire trucks regularly end up stuck at signals or trapped in traffic** when they should be receiving an open corridor to move through.
- The **link between the road and the authorities is broken**. When a stretch of road deteriorates or a blockage appears, it can take **months of daily congestion before it is even reported**. There is **no fast, reliable channel** connecting what is actually happening on the ground to the people who can fix it.

In short, the systems we have are reactive, fragmented and blind to context. What is missing is something that sees traffic coming, understands what kind of traffic it is, and acts before congestion forms.

REAL-WORLD PROBLEM ALIGNMENT

How does your proposed solution fill that gap practically?

We solve this by building an end to end traffic orchestration framework that shifts cities from reacting to congestion after it forms to preventing it before it begins.

- PRAVAH introduces **dynamic traffic lights** at **calculated points along the road, not just at junctions**. These lights work together across the network, **holding and releasing traffic in controlled pulses** so that **bottlenecks downstream get time to clear** before more vehicles arrive. Think of it as putting valves in a water pipe instead of waiting for it to overflow.
- To make these lights intelligent, **we use two types of sensing**. **GNSS data** gives the system a **long range view** of how much traffic is approaching from a distance, while **cameras** at the junction give the **immediate picture** of what is happening right now. Together they give each light **both foresight and awareness**, something no deployed system currently does.
- The same sensing pipeline also **identifies heavy vehicles** so the signal can **adjust its timing to account for their slower startup**, and **detects emergency vehicles** early so a **coordinated green corridor** can be prepared ahead of their arrival without disrupting the rest of the network.
- Beyond control, **PRAVAH monitors road health** by **tracking how speeds change over time on every segment**. When a stretch of road starts **performing poorly for no obvious reason**, it **flags it to the authorities automatically**, closing the gap between the ground reality and the people responsible for fixing it.

The result is a single framework that sees further, thinks ahead, and manages the full network as one connected system instead of a set of disconnected junctions each acting on their own.

REAL-WORLD PROBLEM ALIGNMENT

If possible, mention if it connects to any existing schemes

- **National Urban Transport Policy (NUTP)** – directly calls for ITS and technology-driven urban mobility, exactly what PRAVAH delivers.
- **AMRUT 2.0** – covers 500+ cities with \$35B outlay, funds smart urban infrastructure including traffic systems.
- **IndiaAI Mission** – 10,300+ crore for indigenous AI capabilities and startups, PRAVAH qualifies as applied AI for transport.
- **NHAI ATMS Programme** – government is actively deploying AI-based traffic management on national highways as of 2026, PRAVAH extends this vision into urban networks.
- **iRAD (Integrated Road Accident Database)** – PRAVAH's anomaly detection produces exactly the kind of road intelligence iRAD needs.
- **AIS-140 (Motor Vehicles Act)** – every commercial vehicle already streams GPS to government servers, PRAVAH builds directly on this as its sensing backbone.
- **National Mission on Sustainable Habitat (NMSH)** – targets fuel efficiency and reduced emissions through smarter transport, PRAVAH cuts idling and fuel waste.
- **National Clean Air Programme (NCAP)** – targets 20–30% PM2.5 reduction in 131 cities, less stop-and-go means less pollution.
- **Digital India** – real-time data, connected systems, automated reporting to authorities.
- **Make in India** – indigenous framework built ground-up for Indian traffic, not an imported Western system.

PROPOSED SOLUTION - What are we Building?

PRAVAH treats city traffic like water in a pipe. **Instead of letting vehicles flow continuously** until a bottleneck floods, we **break the flow into controlled pulses**. A **continuous stream of traffic is hard to manage**, but smaller **packets of vehicles are far easier** to time, direct and clear. **Dynamic traffic lights** placed at **calculated distances along the road, not just at junctions, hold and release these packets in sync** so that **downstream points get time to clear** before more traffic arrives.

Each light makes decisions using two eyes. **GNSS data** shows what is **coming from far away**. **Cameras** show what is **happening right now**. Together they give the system both **foresight and immediate awareness**. This same sensing **identifies heavy vehicles** so the **signal adjusts for their slower startup**, **spots emergency vehicles** early to **prepare a green corridor** ahead of them, and **monitors road health** by **tracking speed patterns** over time to **flag problems to authorities automatically**.

The system works with existing infrastructure from day one. As it learns the network, it identifies exactly where placing a new control point would have the highest impact, **so any hardware investment is guided by data, not guesswork**.

One connected network. Every light working in sync. Traffic managed before congestion begins, not after it happens.

PROPOSED SOLUTION - The Digital Twin

PRAVAH runs on a live digital twin of the road network, a continuously updated virtual replica of the real city, built in SUMO and kept in sync with the street through GNSS density and speed on one side and camera queue and vehicle-class data on the other. The twin is not a static map. It mirrors what the network is actually doing, second by second.

- **A mirror of the real road, not a model of an ideal one.** OpenStreetMap gives the twin its geometry, and the fused GNSS and camera feeds keep every segment's density, speed and queue state matched to reality. **The twin always reflects the network as it is, not as it was designed to be.**
- **A place to understand before acting.** The Digital Twin gives PRAVAH the network-wide insights and foresight needed to make better control decisions. It lets the system see how a decision at one junction will affect traffic across the network before that decision reaches the street.
- **The environment where our light-placement model learns.** The twin is also what makes mid-road traffic light placement possible. Our reinforcement learning model treats the twin as its training environment, trying out control points at different positions along the road and watching how each one affects flow. This is how PRAVAH works out where a new light would actually help, before a single one is installed in the real world.
- **Foresight, not just observation.** Because the twin runs faster than real time, PRAVAH can look a few minutes ahead, see a bottleneck forming before it exists on the road, and act early. This is the core shift from reacting to congestion to preventing it.

One virtual city, always in sync with the real one, where every decision is rehearsed before it reaches the road.

PROPOSED SOLUTION - Key Features

- **Traffic Quantisation**
 - PRAVAH **breaks continuous traffic into smaller, timed packets**. Dynamic lights at calculated points hold and release these packets in coordination, **so every part of the network gets only as much traffic as it can handle at that moment**.
- **GNSS + camera fusion**
 - **Cameras see what is right in front. GNSS sees what is coming from kilometres away**. PRAVAH uses both, giving each light the ability to prepare for incoming traffic before it arrives, not just react when it shows up.
- **Heavy vehicle aware control**
 - Trucks and buses take longer to clear a junction than cars. Most signals ignore this. **PRAVAH detects heavy vehicles and adjusts green time so the junction actually clears as intended**, saving fuel and avoiding blockages.
- **Emergency vehicle pre-emption**
 - Instead of clearing one signal when an ambulance is already close, **PRAVAH predicts the route and prepares a green corridor across multiple signals in advance**, letting the vehicle pass without disrupting surrounding traffic.
- **GNSS anomaly detection**
 - PRAVAH tracks speed on every road segment. When a stretch that normally runs at 60 km/h drops to 30 km/h for days, **it flags and reports the issue to authorities automatically**. No extra sensors needed, just existing GNSS data.
- **Zero new hardware to start**
 - PRAVAH runs on cameras and signal infrastructure cities already have, using GNSS data already provided by TomTom floating car data and the data mandated under AIS-140. **It works from day one with no new installations**. As results prove out, **the system identifies where adding mid-road lights would improve control further**, making **hardware investment targeted and data-driven, never speculative**.

PROPOSED SOLUTION - What Makes it Different?

- **Control between junctions, not just at them**
 - **Existing systems (SCATS, SCOOT, SURTRAC) optimise one junction at a time** and hope the next one can handle whatever comes. **PRAVAH controls the full network, breaking continuous flow** into timed packets so bottlenecks never get overwhelmed. **No deployed system does mid-road flow control between junctions.**
- **The Dual Sensor Stack**
 - Current systems use either road sensors (SCATS, SCOOT), or cameras (BATCS), **never together**. GPS data has been used (Google Green Light) but only to advise engineers, not to control signals in real time. PRAVAH uses **long range GNSS** with **local camera data** in one live control loop, **giving every light both distance vision and immediate awareness at the same time.**
- **Signals that know what is in the queue**
 - **No deployed traffic system (SCATS, SCOOT, SURTRAC, BATCS) changes its timing based on whether the queue has trucks or cars.** PRAVAH detects heavy vehicles, accounts for their slower startup, and extends green so the junction actually clears as planned.
- **Green corridors prepared, not scrambled**
 - **Existing emergency systems (Opticom EVP) only react when the vehicle is a few hundred metres from the signal.** **PRAVAH predicts the route and prepares a coordinated corridor** across multiple signals in advance, without collapsing the traffic around it.
- **Detection that acts, not just alerts**
 - Current platforms (Waze, Google Maps) **detect incidents but only alert drivers or authorities.** The information never reaches the traffic controller. PRAVAH **tracks speed anomalies on every road segment** and **feeds that intelligence directly back into control, closing the loop between detection and action.**

TECH STACK & ARCHITECTURE

Tech Stack

- **Simulation:** SUMO + TraCI (Python API)
- **Camera pipeline:** YOLOv8 + ByteTrack for detection, tracking, counting, speed, and vehicle classification
- **GNSS pipeline:** Map-matching (Valhalla or HMM based) to snap GPS points to road segments, then per-segment density and speed estimation from probe traces
- **Control algorithm:** Max-Pressure as the baseline, with GNSS approaching density and camera queue state as separate weighted inputs to the pressure calculation. Custom model using graph neural network based multi-agent RL in PyTorch
- **Anomaly detection:** CUSUM change-point detection on per-segment GNSS speeds
- **Dashboard:** Streamlit or Flask for visualisation
- **Language:** Python

TECH STACK & ARCHITECTURE

APIs/Libraries

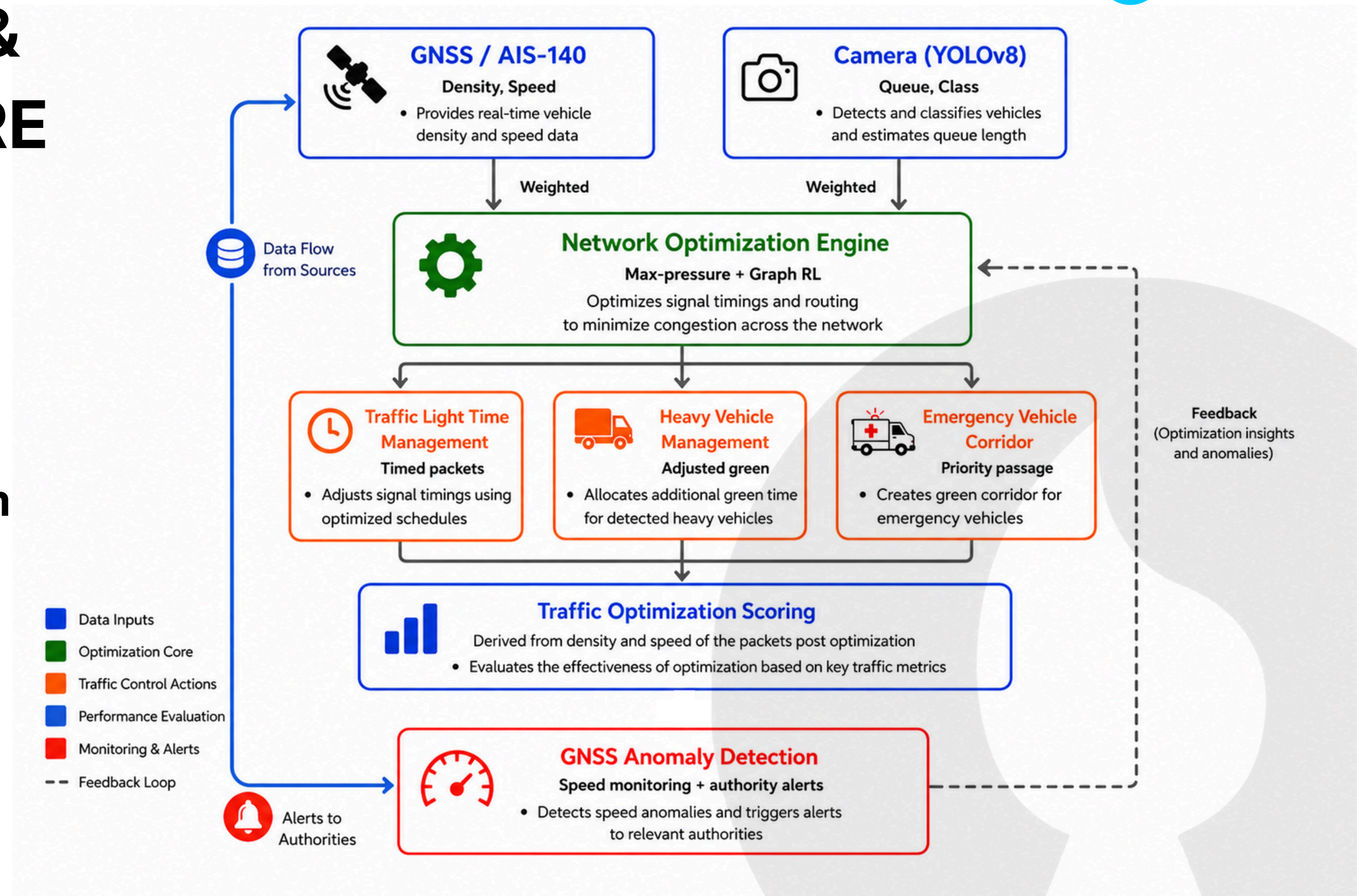
- **APIs**
 - TomTom Traffic Flow API, TomTom Traffic Incidents API, AIS-140 / Vahan NVLT, OpenStreetMap
- **Simulation**
 - SUMO, TraCI
- **Vision**
 - YOLOv8 (Ultralytics), ByteTrack, OpenCV
- **Control**
 - NumPy, PyTorch, PyTorch Geometric, Stable-Baselines3
- **Routing**
 - OSRM
- **Anomaly detection**
 - Ruptures, Pandas
- **Dashboard**
 - Streamlit, Folium, Plotly
- **Utilities**
 - GeoPandas, Shapely, SciPy, Requests

DEPLOYMENT PLAN

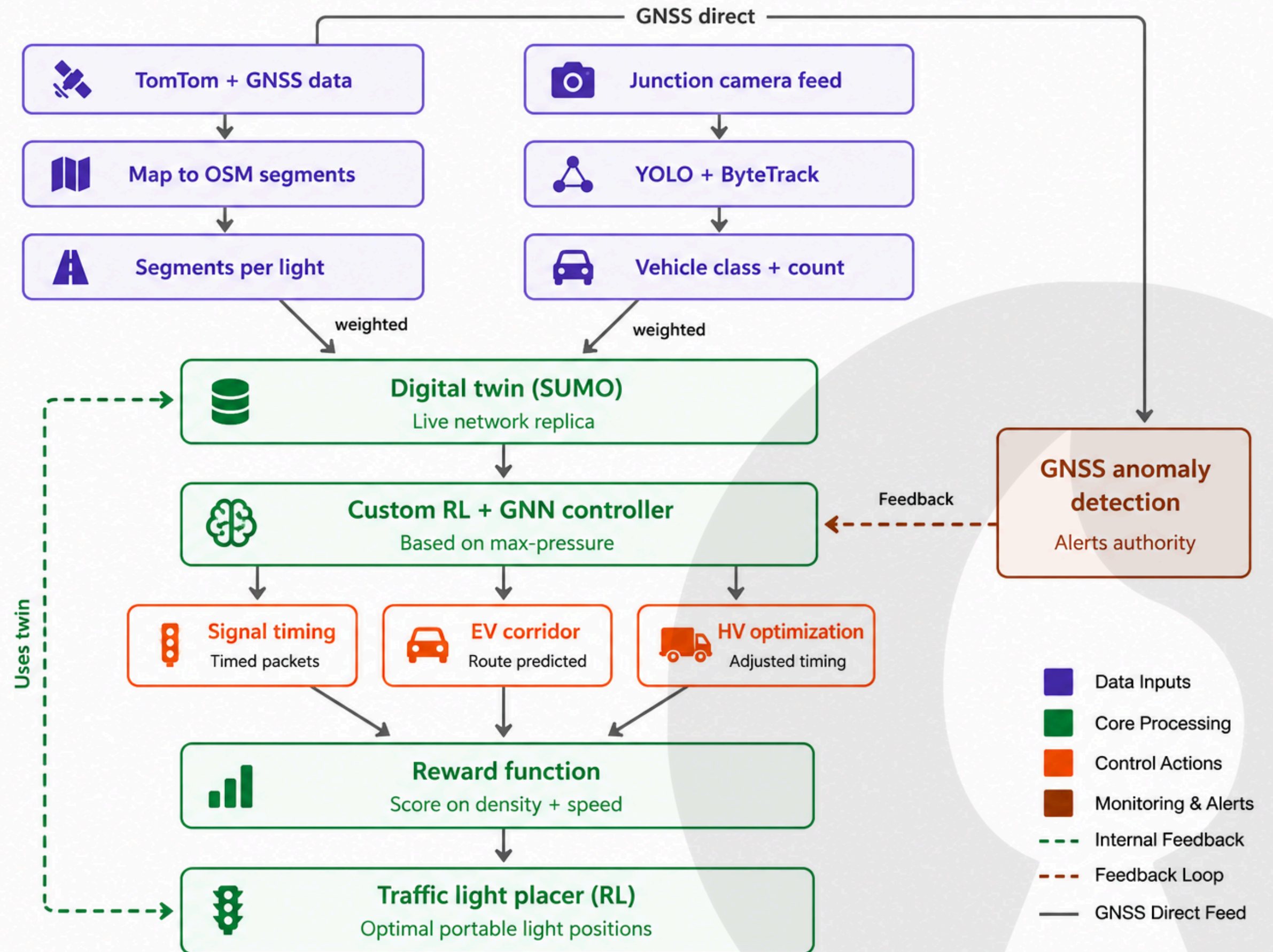
- **Phase 1: Prove the algorithm on existing infrastructure**
 - Select a pilot corridor of 5 to 10 junctions. No new hardware. PRAVAH runs on the cameras already there and pulls GNSS data from AIS-140 feeds and floating car data.
 - **Goal:** prove that fused sensing and coordinated control reduce delay on infrastructure the city already owns.
- **Phase 2: Deploy mid-road control points**
 - Analyse where traffic still builds up between junctions. Install PRAVAH's portable mid-road lights at the positions the algorithm identifies as highest impact.
 - **Goal:** test traffic quantisation in the real world and measure whether mid-road control reduces bottleneck formation beyond simulations.
- **Phase 3: Expand to a wider zone**
 - Extend to multiple corridors and intersecting routes. Activate the full feature set: heavy vehicle aware timing, emergency green corridors, and GNSS anomaly reporting.
 - **Goal:** validate that the system scales from a single corridor to a full network without losing its effectiveness.
- **Phase 4: City scale rollout**
 - Deploy across the city with traffic police and municipal authorities. Each new zone follows the same pattern: algorithm first, hardware where the data justifies it.
 - **Goal:** deliver a fully operational, city wide traffic orchestration system that grows incrementally and proves itself at every step.

TECH STACK & ARCHITECTURE

High-level Block Diagram



ARCHITECTURE DIAGRAM



ARCHITECTURE DIAGRAM

1. Frontend (Visualisation & Dashboard)

- **Framework:** Streamlit / Flask
- **Mapping & Spatial Data:** Folium
- **Analytics & Graphs:** Plotly

2. Backend & Core Execution

- **Core Language:** Python (Unified pipeline)
- **Traffic Simulation Engine:** SUMO (Simulation of Urban MObility)
- **Simulation Interface:** TraCI (Traffic Control Interface - Python API)
- **Routing Engine:** OSRM (Open Source Routing Machine)
- **Data Processing & Geospatial Utilities:** GeoPandas, Shapely, SciPy, Pandas, Requests

ARCHITECTURE DIAGRAM

3. ML Models & Algorithms

- **Computer Vision Pipeline:**
 - **YOLOv8 (Ultralytics):** Object detection and vehicle classification (heavy vehicles, emergency vehicles, cars).
 - **ByteTrack:** Multi-object tracking for vehicle counting, queue state analysis, and speed calculation.
- **Traffic Control Engine:**
 - **Baseline Model:** Max-Pressure Control Algorithm (weighted by GNSS approach density and camera queue state).
 - **Advanced AI Model:** Graph Neural Network (GNN) based Multi-Agent Reinforcement Learning (MARL) built using **PyTorch**, **PyTorch Geometric**, and **Stable-Baselines3**.
- **Anomaly Detection & Map Matching:**
 - **Map Matching:** Valhalla / Hidden Markov Model (HMM) based map-matching to snap GPS traces to road segments.
 - **Anomaly Model:** CUSUM / z-score change-point detection (using **Ruptures** library) for per-segment road health and speed drop analysis.

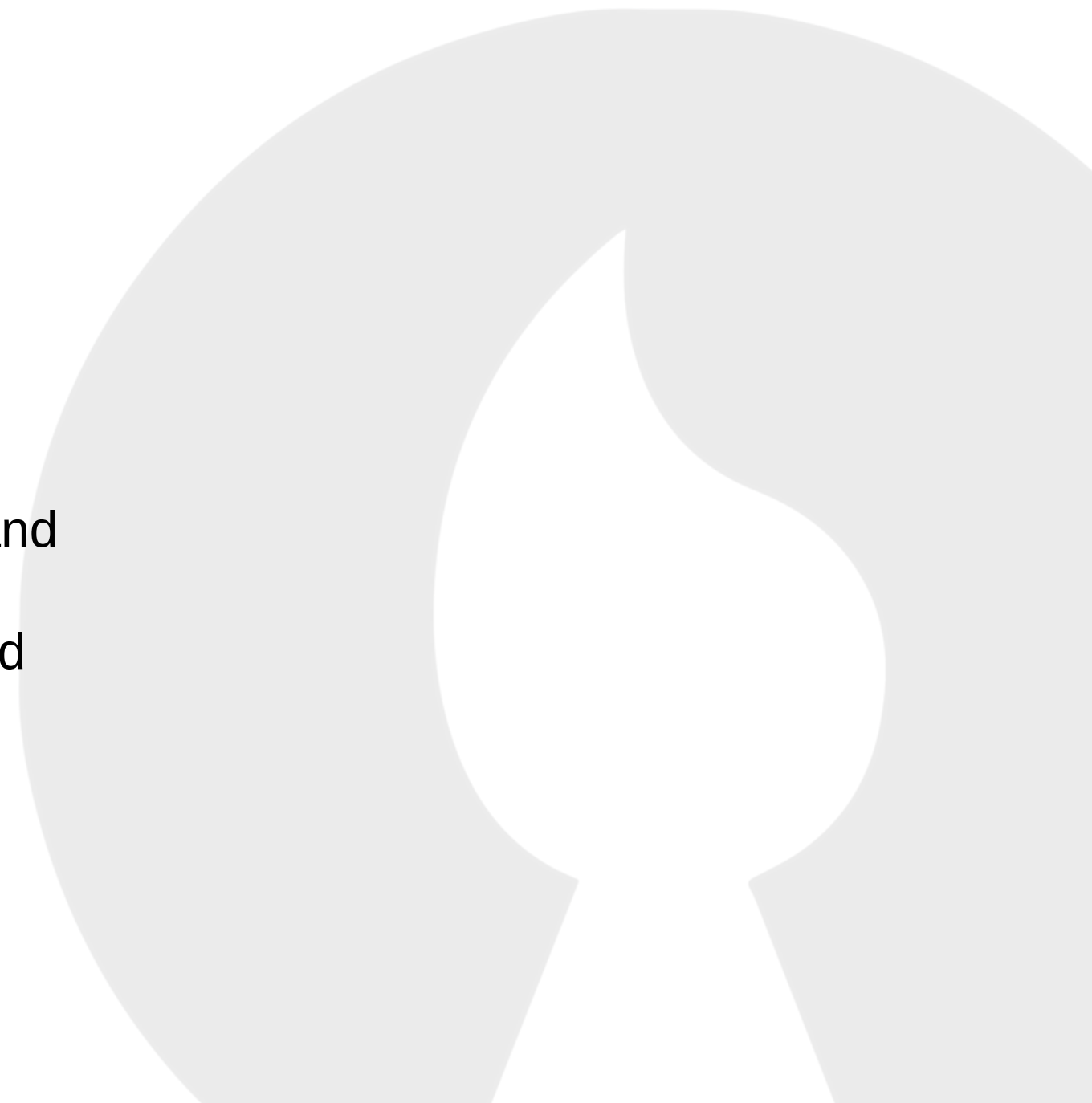
ARCHITECTURE DIAGRAM

4. API Connections & Data Sources

- **TomTom Traffic Flow API:** Real-time traffic speed and density feeds.
- **TomTom Traffic Incidents API:** Live incident and congestion reports.
- **AIS-140 / Vahan NVLT:** Direct stream of commercial vehicle GPS feeds (Government motor vehicles framework).
- **OpenStreetMap (OSM):** Road network geometry and map layers.

5. Cloud, Storage & Data Management

- **Data Processing Layer:** In-memory geospatial processing via **GeoPandas** and **Pandas** for live probe trace streaming.
- **Geospatial Storage:** OpenStreetMap road network graph representations and segment-level speed history tables.



THANK
YOU

